

Lesson 2: Programming with Numbers and Strings

Faculty of Data Science & Artificial Intelligence (FDA) - National Economics University

Course Code: DSAI1003

Contents

Lesson 2. Programming with Numbers and Strings	1
Lesson Introduction	1
Knowledge and Skills to Be Achieved	2
Lesson Structure	2
2.1 Variables	2
2.1.1 Defining Variables	3
Assignment Is Not Mathematical Equality	4
2.1.2 Number Types	4
2.1.3 Variable Names	5
Rules	5
Descriptive Names	5
2.1.4 Constants	5
2.1.5 Comments	6
Example – Computing Soda Volume	6
Common Error 2.1 – Using Undefined Variables	6
Why Does This Error Occur?	7
Programming Tip 2.1 – Choose Descriptive Variable Names	7
Programming Tip 2.2 – Do Not Use Magic Numbers	7
Self Check 2.1	8
Question 1	8
Question 2	8
Question 3	8
Question 4	8
Question 5	8
2.2 Arithmetic	9
2.2.1 Basic Arithmetic Operations	9
Operator Precedence	9
Mixing int and float	9
2.2.2 Powers	10
2.2.3 Floor Division and Remainder	10
Floor Division //	10
Remainder %	11
Example – Converting Pennies to Dollars and Cents	11
2.2.4 Calling Functions	11
2.2.5 Mathematical Functions and the math Module	12
Common Error 2.2 – Roundoff Errors	12
Common Error 2.3 – Unbalanced Parentheses	13

Programming Tip 2.3 – Use Spaces in Expressions	13
Special Topic 2.1 – Other Ways to Import Modules	13
Special Topic 2.2 – Combining Assignment and Arithmetic	13
Special Topic 2.3 – Line Joining	14
Self Check 2.2	14
Question 1	14
Question 2	14
Question 3	14
Question 4	14
Question 5	15
2.3 Problem Solving: First Do It By Hand	15
Example – Alternating Tiles Along a Wall	15
General Algorithm	16
Worked Example 2.1 – Computing Travel Time	16
Segment 1	16
Segment 2	16
Self Check 2.3	17
Question 1	17
Question 2	17
Question 3	17
2.4 Strings	17
2.4.1 The String Type	17
String Length	18
2.4.2 Concatenation and Repetition	18
Concatenation +	18
Repetition *	18
2.4.3 Converting Between Numbers and Strings	18
Number → String	18
String → Integer	19
String → Floating-Point Number	19
2.4.4 Strings and Characters	19
Example – Building Initials	19
2.4.5 String Methods	20
Special Topic 2.4 – Character Values	20
Special Topic 2.5 – Escape Sequences	20
Computing & Society 2.1 – International Alphabets and Unicode	21
Self Check 2.4	21
Question 1	21
Question 2	21
Question 3	22
Question 4	22
Question 5	22
2.5 Input and Output	22
2.5.1 User Input	22
2.5.2 Numerical Input	23
2.5.3 Formatted Output	23
Floating-Point Values	23
Field Width	23

Common Format Specifiers	23
Formatting Multiple Values	24
Example – Price per Ounce	24
Programming Tip 2.4 – Don’t Wait to Convert	24
HOW TO 2.1 – Writing Simple Programs	25
Problem – Vending Machine Change	25
Step 1. Understand the Inputs and Outputs	25
Inputs	25
Outputs	25
Step 2. Work Out an Example by Hand	25
Step 3. Write Pseudocode	26
Step 4. Identify Variables and Constants	26
Step 5. Translate the Computation into Python	26
Step 6. Add Input and Output	26
Step 7. Assemble the Program	26
Worked Example 2.2 – Computing the Cost of Stamps	27
Input	27
Outputs	27
Hand Calculation for \$1	27
General Computation	27
Computing & Society 2.2 – Bugs in Silicon	28
Self Check 2.5	28
Question 1	28
Question 2	28
Question 3	28
Question 4	29
Question 5	29
TOOLBOX 2.1 – Symbolic Processing with SymPy	29
Expanding Expressions	29
Solving Equations	29
Differentiation	29
Integration	29
Substitution and Numerical Evaluation	29
Plotting	30
Chapter 2 Summary	30
Variables	30
Arithmetic	30
Problem Solving	30
Strings	31
Input and Output	31
Graphics	31
Summary Quiz	31
Question 1	31
Question 2	31
Question 3	31
Question 4	32
Question 5	32
Question 6	32
Question 7	32
Question 8	33
Question 9	33

Question 10	33
Review Exercises	33
Exercise 1. Trace Assignments	33
Exercise 2. Translate Mathematics into Python	34
Exercise 3. Floor Division and Remainder	34
Exercise 4. String Expressions	34
Exercise 5. Explain the Difference	35
Exercise 6. Roundoff Error	35
Practice Exercises	35
Exercise 1. Paper Dimensions	35
Exercise 2. Powers of a Number	35
Exercise 3. Two Integers	36
Exercise 4. Split a Five-Digit Integer	36
Exercise 5. First, Middle, and Last Character	36
Exercise 6. Monogram	36
Exercise 7. Vending Machine	36
Exercise 8. Temperature and Dew Point	37
Open Exercises	37
Exercise 1	37
Exercise 2	37
Exercise 3	37
Exercise 4	37
Key Terms	37
Study Guidance	38

Lesson 2. Programming with Numbers and Strings

Last updated: September 11, 2026

Source: Chapter 2 – *Programming with Numbers and Strings*, in *Python for Everyone, 3rd Edition* by Cay Horstmann and Rance Necaise.

This lecture preserves the structure, terminology, examples, and instructional focus of Chapter 2, while reorganizing the material for direct classroom use. The examples intentionally follow the conventions used in the textbook, including its variable-naming style and the % string-format operator.

Lesson Introduction

Chapter 1 introduced the idea of a computer program, the basic organization of a computer, the Python programming environment, and the role of algorithms. Chapter 2 begins the transition from *understanding what programs are* to *writing programs that perform useful computations*.

The main building blocks introduced in this chapter are:

1. **Variables** for storing values.
2. **Numeric types** for representing integers and floating-point values.
3. **Arithmetic expressions** for computing new values.
4. **Strings** for representing and manipulating text.
5. **Input and output** for communicating with the program user.
6. **Simple graphics** for producing drawings with geometric shapes and text.

A recurring theme throughout the chapter is that a program should not be written by immediately typing Python statements. A better workflow is:

```
Understand the problem
↓
Work out a concrete example by hand
↓
Identify variables and constants
↓
Develop the computation / pseudocode
↓
Translate the solution into Python
↓
Run, inspect, and improve the result
```

Knowledge and Skills to Be Achieved

After completing this lesson, students will be able to:

- Define and use **variables** and **constants**.
 - Explain the difference between `int` and `float` values.
 - Apply the assignment operator `=` correctly.
 - Choose valid and descriptive variable names.
 - Use comments to explain program intent.
 - Write arithmetic expressions using `+`, `-`, `*`, `/`, `//`, `%`, and `**`.
 - Explain operator precedence and the role of parentheses.
 - Call built-in functions such as `abs()`, `round()`, `min()`, and `max()`.
 - Import and use functions from the `math` module.
 - Recognize floating-point **roundoff errors**.
 - Perform a computation by hand before programming it.
 - Create and manipulate **strings**.
 - Use string concatenation, repetition, indexing, and methods.
 - Convert between strings and numbers using `str()`, `int()`, and `float()`.
 - Use `input()` to obtain user input.
 - Format numeric and string output using the textbook's `%` formatting operator.
 - Write small programs according to an **Input** → **Process** → **Output** structure.
 - Create simple graphical drawings using a graphics window and canvas.
 - Decompose a drawing into lines, rectangles, ovals, and text.
-

Lesson Structure

1. Variables
 2. Arithmetic
 3. Problem Solving: First Do It By Hand
 4. Strings
 5. Input and Output
 6. Graphics: Simple Drawings
 7. Optional Toolbox: Symbolic Processing with SymPy
 8. Summary and Exercises
-

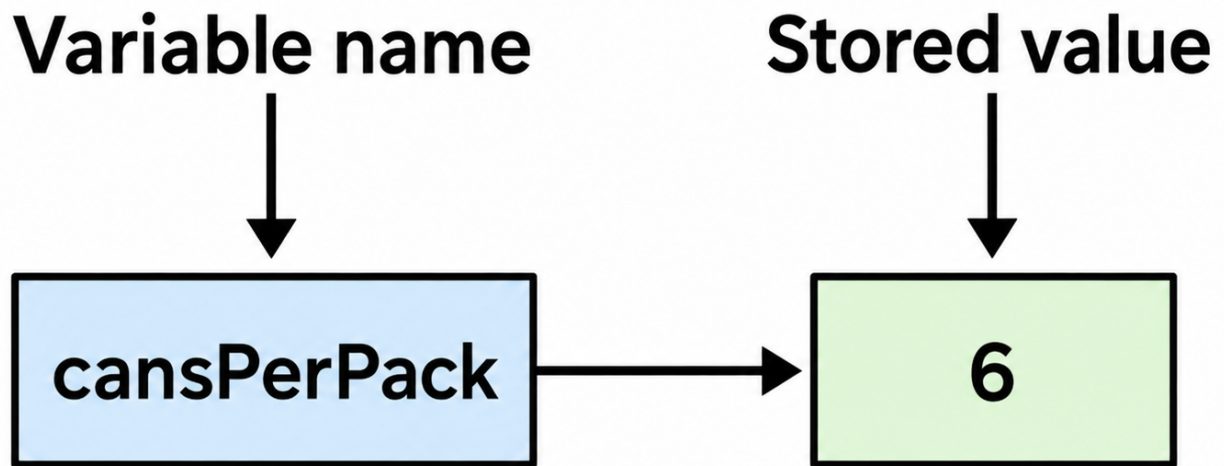
2.1 Variables

A program becomes useful when it can store values, modify them, and use them in later computations.

A **variable** is a storage location in a computer program. Each variable has:

- a **name**;
- a **value** currently stored in that location.

A useful analogy is a labeled parking space:



The name identifies the storage location; the value is its current content.

2.1.1 Defining Variables

A value is stored in a variable with an **assignment statement**.

```
cansPerPack = 6
```

The general form is:

```
variableName = value
```

The first time a variable is assigned a value, the variable is **created and initialized**.

```
cansPerPack = 6  
print(cansPerPack)
```

Output:

```
6
```

If the variable is assigned another value later, the previous value is replaced.

```
cansPerPack = 6  
cansPerPack = 8  
print(cansPerPack)
```

Output:

8

Assignment Is Not Mathematical Equality

The symbol = in Python is the **assignment operator**. It does not mean mathematical equality.

Consider:

```
cansPerPack = cansPerPack + 2
```

If cansPerPack currently contains 8, Python performs the following steps:

1. Read the current value: 8
2. Compute $8 + 2 = 10$
3. Store 10 back into cansPerPack

After the statement executes:

```
cansPerPack == 10
```

conceptually, the variable now stores 10.

This is perfectly meaningful in programming, even though the algebraic statement $x = x + 2$ would be impossible as an equation.

2.1.2 Number Types

Every value in Python has a **data type**. The data type determines:

- how the value is represented;
- which operations can be performed on the value.

Two important numeric types are:

Python type	Meaning	Examples
int	integer / whole number	6, 0, -25
float	floating-point number	0.355, 1.0, -3.5

Examples:

```
count = 6  
price = 2.95
```

A numeric value written directly in a program is called a **number literal**.

```
6  
0.5  
1.0  
1E6  
2.96E-2
```

Important observations:

- 6 is an int.
- 6.0 is a float.

- Numbers written in exponential notation such as 1E6 are floating-point values.

The type is associated with the **value**, not permanently with the variable name.

```
taxRate = 5
taxRate = 5.5
```

Python allows this. However, the chapter recommends keeping the type of a variable conceptually consistent. If a variable represents a tax rate, initialize it as a floating-point value when fractional values are possible:

```
taxRate = 5.0
```

2.1.3 Variable Names

Variable names must follow Python's naming rules.

Rules

1. A name must start with a letter or underscore `_`.
2. Remaining characters may be letters, digits, or underscores.
3. Spaces and symbols such as `?`, `%`, `.`, and `/` are not allowed.
4. Names are **case sensitive**.
5. Reserved words such as `if` and `class` cannot be used as variable names.

Examples:

Name	Valid?	Comment
canVolume1	Yes	Letters and digits are allowed.
x	Yes	Legal, but often not descriptive enough.
CanVolume	Yes	Different from <code>canVolume</code> ; violates the book's lowercase-variable convention.
6pack	No	Cannot start with a digit.
can volume	No	Cannot contain spaces.
class	No	Reserved word.
ltr/fl.oz	No	Contains <code>/</code> and <code>..</code>

Descriptive Names

Compare:

```
cv = 0.355
```

with:

```
canVolume = 0.355
```

The second name gives much more information about the meaning of the variable.

2.1.4 Constants

A **constant** is a value that should remain unchanged throughout the program.

Python does not technically prevent a programmer from changing a constant. Instead, the chapter uses the convention of writing constant names in uppercase letters.

```
BOTTLE_VOLUME = 2.0
MAX_SIZE = 100
```

Named constants make programs easier to understand.

Less clear:

```
totalVolume = bottles * 2
```

Clearer:

```
BOTTLE_VOLUME = 2.0
totalVolume = bottles * BOTTLE_VOLUME
```

The named constant explains *why* the value 2.0 appears in the computation.

2.1.5 Comments

A **comment** is an explanation written for human readers.

```
CAN_VOLUME = 0.355 # Liters in a 12-ounce can
```

Python ignores everything from # to the end of the line.

A file-level comment can explain the purpose of a program:

```
##
# This program computes the volume, in liters, of a six-pack of soda cans.
#
```

Comments are particularly useful for explaining:

- program purpose;
 - non-obvious constants;
 - important assumptions;
 - major steps of a computation.
-

Example – Computing Soda Volume

```
##
# This program computes the volume of a six-pack of soda cans
# and the total volume when a two-liter bottle is added.
#

CAN_VOLUME = 0.355
BOTTLE_VOLUME = 2.0

cansPerPack = 6

totalVolume = cansPerPack * CAN_VOLUME
print("A six-pack of 12-ounce cans contains", totalVolume, "liters.")

totalVolume = totalVolume + BOTTLE_VOLUME
print("A six-pack and a two-liter bottle contain", totalVolume, "liters.")
```

Conceptually:

```
6 cans × 0.355 L/can = 2.13 L
2.13 L + 2.00 L      = 4.13 L
```

Common Error 2.1 – Using Undefined Variables

A variable must be created before it is used.

Incorrect:

```
canVolume = 12 * literPerOunce
literPerOunce = 0.0296
```

When Python executes the first statement, `literPerOunce` does not yet exist.

Python therefore raises a **NameError**:

```
NameError: name 'literPerOunce' is not defined
```

Why Does This Error Occur?

Python executes statements **from top to bottom**.

At this point:

```
canVolume = 12 * literPerOunce
```

Python tries to find the current value of `literPerOunce`, but that variable has not yet been assigned a value.

Therefore, the name is **undefined**.

Correct:

```
literPerOunce = 0.0296
canVolume = 12 * literPerOunce
```

Program statements are executed in order, so definitions must appear before uses.

Programming Tip 2.1 – Choose Descriptive Variable Names

Prefer:

```
canVolume = 0.355
```

over:

```
cv = 0.355
```

Descriptive names reduce the mental effort required to understand a program and make code easier to maintain.

Programming Tip 2.2 – Do Not Use Magic Numbers

A **magic number** is an unexplained numeric literal in a program.

Less clear:

```
totalVolume = bottles * 2
```

Better:

```
BOTTLE_VOLUME = 2.0
totalVolume = bottles * BOTTLE_VOLUME
```

Using named constants makes changes safer. If the bottle size changes, only the constant definition needs to be updated.

Self Check 2.1

Question 1

What happens the first time this statement is executed?

```
count = 10
```

Answer

A variable named count is created and initialized with the integer value 10.

Question 2

Suppose:

```
x = 7
x = x + 3
```

What value does x contain afterwards?

Answer

10.

Python first evaluates $x + 3$, then stores the result back into x.

Question 3

What is the difference between 6 and 6.0?

Answer

6 has type int; 6.0 has type float.

Question 4

Which name best represents a constant?

- A. bottleVolume
- B. BOTTLE_VOLUME
- C. bottle volume
- D. 2liters

Answer

B. BOTTLE_VOLUME

The chapter uses uppercase names for constants.

Question 5

Why is the following code incorrect?

```
result = price * quantity
quantity = 4
```

Answer

quantity is used before it is created and initialized.

2.2 Arithmetic

Python can be used as a calculator, but arithmetic expressions must be written using Python's operators and syntax.

2.2.1 Basic Arithmetic Operations

The basic operators are:

Operation	Python operator	Example
Addition	+	<code>a + b</code>
Subtraction	-	<code>a - b</code>
Multiplication	*	<code>a * b</code>
Division	/	<code>a / b</code>

For example, the mathematical expression

$$\frac{a + b}{2}$$

is written in Python as:

```
(a + b) / 2
```

The combination of literals, variables, operators, and parentheses is called an **expression**.

Operator Precedence

Python follows precedence rules similar to algebra:

1. Parentheses
2. Exponentiation
3. Multiplication and division
4. Addition and subtraction

Example:

```
a + b / 2
```

computes `b / 2` first.

To average `a` and `b`, use:

```
(a + b) / 2
```

Operators of the same precedence are generally evaluated from left to right.

```
10 - 2 - 3
```

is evaluated as:

```
(10 - 2) - 3 = 5
```

Mixing `int` and `float`

If integer and floating-point values are mixed in an arithmetic expression, the result is a floating-point value.

```
7 + 4.0
```

Result:

11.0

2.2.2 Powers

Python uses `**` for exponentiation.

```
2 ** 3
```

means:

$$2^3 = 8$$

Example:

```
10 * 2 ** 3
```

Result:

```
80
```

Exponentiation has higher precedence than multiplication.

Power operations are evaluated from right to left:

```
10 ** 2 ** 3
```

is interpreted as:

```
10 ** (2 ** 3)
```

not:

```
(10 ** 2) ** 3
```

A mathematical expression such as

$$b \left(1 + \frac{r}{100}\right)^n$$

becomes:

```
b * (1 + r / 100) ** n
```

2.2.3 Floor Division and Remainder

Normal division `/` produces a floating-point result.

```
7 / 4
```

Result:

```
1.75
```

Floor Division `//`

For positive integers, `//` computes the quotient and discards the fractional part.

```
7 // 4
```

Result:

1

Remainder %

The % operator computes the remainder.

```
7 % 4
```

Result:

```
3
```

Example – Converting Pennies to Dollars and Cents

```
pennies = 1729

dollars = pennies // 100
cents = pennies % 100

print(dollars)
print(cents)
```

Output:

```
17
29
```

Useful patterns for a positive integer n:

```
n % 10      # last digit
n // 10     # all digits except the last
n % 100     # last two digits
n % 2       # 0 if even, 1 if odd
```

2.2.4 Calling Functions

A function may **return a value** that can be used in an expression, printed, or stored in a variable.

Example:

```
distance = abs(-173)
```

abs() returns the absolute value.

Other useful built-in mathematical functions include:

Function	Purpose
abs(x)	Absolute value of x
round(x)	Round x to a whole number
round(x, n)	Round x to n decimal places
max(x1, ..., xn)	Largest argument
min(x1, ..., xn)	Smallest argument

Examples:

```
print(abs(-10))
print(round(7.627, 2))
print(min(7.25, 10.95, 5.95, 6.05))
```

A function must be called with an appropriate number of arguments.

Incorrect:

```
abs()
abs(-10, 2)
```

The `abs()` function expects exactly one argument.

2.2.5 Mathematical Functions and the math Module

Python's standard library contains reusable modules.

The `math` module provides many mathematical functions.

To use `sqrt()`:

```
from math import sqrt

y = sqrt(25)
print(y)
```

Output:

```
5.0
```

Selected functions include:

Function	Meaning
<code>sqrt(x)</code>	Square root
<code>trunc(x)</code>	Truncate floating-point value to an integer
<code>sin(x)</code>	Sine of x in radians
<code>cos(x)</code>	Cosine of x in radians
<code>tan(x)</code>	Tangent of x in radians
<code>exp(x)</code>	e^x
<code>degrees(x)</code>	Radians $\rightarrow$ degrees
<code>radians(x)</code>	Degrees $\rightarrow$ radians
<code>log(x)</code>	Natural logarithm
<code>log(x, base)</code>	Logarithm with specified base

The module also provides constants such as `pi`.

```
from math import pi

area = pi * radius ** 2
```

Common Error 2.2 – Roundoff Errors

Floating-point numbers cannot always represent decimal values exactly.

Example:

```
price = 4.35
quantity = 100
total = price * quantity
print(total)
```

A computer may display a value close to, but not exactly, 435.

This is not necessarily a programming mistake. It is a consequence of representing many decimal fractions using a finite number of binary digits.

For user-facing output, round the value or format it with a fixed number of decimal places.

Common Error 2.3 – Unbalanced Parentheses

Consider:

```
((a + b) * t / 2 * (1 - t))
```

There are more opening parentheses than closing parentheses.

A useful checking rule is:

- while scanning left to right, the number of closing parentheses should never exceed the number of opening parentheses;
 - at the end, the counts must match.
-

Programming Tip 2.3 – Use Spaces in Expressions

Prefer:

```
x1 = (-b + sqrt(b ** 2 - 4 * a * c)) / (2 * a)
```

over:

```
x1=(-b+sqrt(b**2-4*a*c))/(2*a)
```

Spaces make the structure of an expression easier to read.

Special Topic 2.1 – Other Ways to Import Modules

Import selected functions:

```
from math import sqrt, sin, cos
```

Import all names from a module:

```
from math import *
```

Import the module itself:

```
import math
```

```
y = math.sqrt(x)
```

The last style makes the origin of the function explicit.

Special Topic 2.2 – Combining Assignment and Arithmetic

Python provides augmented assignment operators.

```
total += cans
```

is equivalent to:

```
total = total + cans
```

Similarly:

```
total *= 2
```

```
count += 1
```

Special Topic 2.3 – Line Joining

Long expressions can be split across lines when the break occurs inside parentheses.

```
x1 = ((-b + sqrt(b ** 2 - 4 * a * c))  
      / (2 * a))
```

This is preferable to breaking a complete expression at an arbitrary point.

Self Check 2.2

Question 1

What is the value of:

```
10 + 6 / 2
```

Answer

13.0

Division is performed before addition.

Question 2

What is the result of:

```
17 // 5
```

```
17 % 5
```

Answer

3

2

Question 3

What is the last digit of a positive integer n?

Answer

```
n % 10
```

Question 4

Why is the following expression different from $(a + b) / 2$?

```
a + b / 2
```

Answer

Because division has higher precedence than addition. Python computes $b / 2$ first, then adds a .

Question 5

Which import is required before using `sqrt(x)` in the textbook style?

Answer

```
from math import sqrt
```

2.3 Problem Solving: First Do It By Hand

Before translating a problem into Python, work out at least one concrete example manually.

The principle is:

If you cannot compute a solution yourself, it is unlikely that you can write a reliable program that automates the computation.

Example – Alternating Tiles Along a Wall

Suppose black and white tiles are placed along a wall with the requirements:

- the first tile is black;
- colors alternate;
- the last tile must also be black.

Assume:

```
Total width = 100 inches  
Tile width  = 5 inches
```

If we use 20 tiles, the last tile would be white. Instead, think of the pattern as:

```
B | WB | WB | WB | ... | WB
```

There is one initial black tile, followed by black/white pairs.

The first tile uses 5 inches, leaving 95 inches.

Each pair uses:

```
2 × 5 = 10 inches
```

Number of complete pairs:

```
95 // 10 = 9
```

Number of tiles:

```
1 + 2 × 9 = 19
```

The tiles occupy:

```
19 × 5 = 95 inches
```

Total unused space:

```
100 - 95 = 5 inches
```

Gap at each end:

$5 / 2 = 2.5$ inches

General Algorithm

```
number of pairs = integer part of
                  (total width - tile width) / (2 × tile width)

number of tiles = 1 + 2 × number of pairs

gap at each end =
                  (total width - number of tiles × tile width) / 2
```

Doing the arithmetic by hand exposes the structure needed for the algorithm.

Worked Example 2.1 – Computing Travel Time

A robot travels toward an item. It can move faster on a road than across rocky terrain.

Inputs:

- horizontal distance to the item;
- vertical distance to the item;
- speed on the road;
- speed on rocky terrain;
- length of the first road segment.

The total travel time consists of two parts.

Segment 1

```
time1 = length1 / speed1
```

Segment 2

The second segment is the hypotenuse of a right triangle.

$$\text{segment2Length} = \sqrt{(\text{xDistance} - \text{segment1Length})^2 + \text{yDistance}^2}$$

Then:

```
time2 = segment2Length / speed2
```

Total:

```
totalTime = time1 + time2
```

Python version:

```
from math import sqrt

segment1Time = segment1Length / segment1Speed
segment2Length = sqrt((xDistance - segment1Length) ** 2 + yDistance ** 2)
segment2Time = segment2Length / segment2Speed
totalTime = segment1Time + segment2Time
```

The worked example also reinforces the use of descriptive variable names in the final program.

Self Check 2.3

Question 1

Why should a programmer work out a concrete example by hand before coding?

Answer

Because a hand calculation helps reveal the required steps and exposes logical errors before Python syntax is introduced.

Question 2

In the tile example, why is floor division useful?

Answer

Because only complete tile pairs can be placed. A fractional pair is not meaningful.

Question 3

Which order is preferable?

- A. Write code → guess the algorithm → test
- B. Work out an example → derive the algorithm → write code
- C. Write code → choose variable names → understand the problem

Answer

B.

2.4 Strings

Many programs process text rather than numbers.

A **string** is a sequence of characters.

Examples:

```
"Hello"  
"Python"  
"123 Main Street"
```

Characters include:

- letters;
 - digits;
 - punctuation;
 - spaces;
 - Unicode symbols.
-

2.4.1 The String Type

A string can be stored in a variable.

```
greeting = "Hello"  
print(greeting)
```

A **string literal** is a specific string written directly in source code.

Python supports both single and double quotation marks:

```
"This is a string."  
'So is this.'
```

This allows one quotation style to appear inside the other:

```
message = 'He said "Hello"'
```

String Length

Use `len()` to obtain the number of characters.

```
length = len("World!")
```

`length` becomes 6.

Spaces count as characters.

The **empty string** contains zero characters:

```
""  
,
```

2.4.2 Concatenation and Repetition

Concatenation +

The `+` operator joins strings.

```
firstName = "Harry"  
lastName = "Morgan"  
name = firstName + " " + lastName
```

Result:

```
Harry Morgan
```

Both operands must be strings.

Incorrect:

```
"Agent " + 1729
```

Repetition *

A string can be repeated by an integer factor.

```
dashes = "-" * 50
```

Another example:

```
message = "Echo. "  
print(message * 5)
```

2.4.3 Converting Between Numbers and Strings

Number → String

Use `str()`.

```
id = 1729  
name = "Agent " + str(id)
```

Result:

```
Agent 1729
```

String → Integer

Use `int()`.

```
id = int("1729")
```

String → Floating-Point Number

Use `float()`.

```
price = float("17.29")
```

The string must represent a valid numeric literal for the requested conversion.

2.4.4 Strings and Characters

A string is a sequence of characters. Each character has an **index**.

Python starts counting at 0.

```
String: H a r r y
Index:  0 1 2 3 4
```

Given:

```
name = "Harry"
```

we can access individual characters:

```
first = name[0]
last = name[4]
```

Results:

```
first = "H"
last = "y"
```

The last valid index is:

```
len(name) - 1
```

A general way to get the last character is:

```
last = name[len(name) - 1]
```

Using an index outside the valid range causes an exception at run time.

Example – Building Initials

```
first = "Rodolfo"
second = "Sally"
initials = first[0] + "&" + second[0]
print(initials)
```

Output:

```
R&S
```

2.4.5 String Methods

A **method** is an operation associated with an object.

For a string object:

```
name = "John Smith"
```

we can call:

```
uppercaseName = name.upper()
lowercaseName = name.lower()
```

Useful methods:

Method	Result
s.lower()	Lowercase version of s
s.upper()	Uppercase version of s
s.replace(old, new)	New string with replacements

Example:

```
name2 = name.replace("John", "Jane")
```

Result:

```
Jane Smith
```

These methods return new strings. They do not modify the original string object.

Special Topic 2.4 – Character Values

Characters are represented internally by integer codes.

Use `ord()` to obtain the code of a character:

```
ord("H")
```

Use `chr()` to convert a numeric code back to a character:

```
chr(97)
```

Examples:

```
print("The letter H has code", ord("H"))
print("Code 97 represents", chr(97))
```

Special Topic 2.5 – Escape Sequences

An **escape sequence** begins with a backslash and represents a special character inside a string.

Examples:

```
"You're \"Welcome\""
```

```
"C:\\Temp\\Secret.txt"
```

```
print("*\n**\n***")
```

Output:

```
*  
**  
***
```

Common sequences:

Escape sequence	Meaning
\"	Double quotation mark inside a double-quoted string
\\	Backslash
\n	Newline

Computing & Society 2.1 – International Alphabets and Unicode

Text processing is not limited to the English alphabet.

Different writing systems include:

- accented European alphabets;
- Greek;
- Cyrillic;
- Hebrew;
- Arabic;
- Chinese characters;
- Japanese and Korean writing systems;
- symbols and emoji.

Unicode provides a common encoding system capable of representing characters from writing systems around the world.

Python 3 strings support Unicode.

Conceptually:

```
text = "£100"  
print(text[0])
```

Each string position corresponds to a Unicode character, not simply an ASCII byte.

Self Check 2.4

Question 1

What is the value of:

```
len("Hello")
```

Answer

5

Question 2

What is the value of:

```
"Py" + "thon"
```

Answer

Python

Question 3

What does this produce?

```
"_ " * 5
```

Answer

Question 4

Given:

```
s = "Python"
```

what are `s[0]` and `s[len(s) - 1]`?

Answer

```
s[0]          → "p"
s[len(s) - 1] → "n"
```

Question 5

Why does this fail?

```
"Agent " + 1729
```

Answer

String concatenation requires strings on both sides. Convert the integer first:

```
"Agent " + str(1729)
```

2.5 Input and Output

Most useful programs receive data from a user and produce results based on those inputs.

A common structure is:

```
Input
↓
Process
↓
Output
```

2.5.1 User Input

Use `input()` to read text from the keyboard.

```
first = input("Enter your first name: ")
```

The string passed to `input()` is the **prompt**.

Conceptually, `input()` performs these steps:

```
Display prompt
↓
Wait for keyboard input
↓
User presses Enter
↓
Return entered characters as a string
```

Example:

```
first = input("Enter your first name: ")
second = input("Enter your significant other's first name: ")

initials = first[0] + "&" + second[0]
print(initials)
```

2.5.2 Numerical Input

`input()` always returns a string.

To read an integer:

```
userInput = input("Please enter the number of bottles: ")
bottles = int(userInput)
```

To read a floating-point value:

```
userInput = input("Enter price per bottle: ")
price = float(userInput)
```

These operations can also be combined:

```
bottles = int(input("Please enter the number of bottles: "))
price = float(input("Enter price per bottle: "))
```

2.5.3 Formatted Output

The chapter uses Python's % **string format operator** to control how values are displayed.

Floating-Point Values

```
price = 1.215962441314554
print("%.2f" % price)
```

Output:

```
1.22
```

Field Width

```
print("%10.2f" % price)
```

The value occupies a field of width 10 and is right-justified.

Common Format Specifiers

Specifier	Meaning
%d	Integer
%f	Floating-point value
%.2f	Float with 2 digits after decimal point
%7.2f	Width 7, 2 decimal places
%s	String
%9s	Right-justified string in width 9
%-9s	Left-justified string in width 9
%%	Literal percent sign
%+5d	Show sign for positive/negative integer

Formatting Multiple Values

```
quantity = 24
total = 17.29

print("Quantity: %d Total: %10.2f" % (quantity, total))
```

Values are matched to format specifiers from left to right.

Example – Price per Ounce

```
##
# This program prints the price per ounce for a six-pack of cans.
#

CANS_PER_PACK = 6

packPrice = float(input("Please enter the price for a six-pack: "))
canVolume = float(input("Please enter the volume for each can (in ounces): "))

packVolume = canVolume * CANS_PER_PACK
pricePerOunce = packPrice / packVolume

print("Price per ounce: %8.2f" % pricePerOunce)
```

This program demonstrates the full **Input** → **Process** → **Output** pattern.

Programming Tip 2.4 – Don't Wait to Convert

When reading numeric input, convert it immediately.

Less desirable:

```
unitPrice = input("Enter the unit price: ")
price1 = float(unitPrice)
price2 = 12 * float(unitPrice)
```

Better:

```
unitPriceInput = input("Enter the unit price: ")
unitPrice = float(unitPriceInput)
```

```
price1 = unitPrice  
price2 = 12 * unitPrice
```

Or combine the calls:

```
unitPrice = float(input("Enter the unit price: "))
```

Immediate conversion reduces repetition and makes it less likely that a string will accidentally be used in a numerical computation.

HOW TO 2.1 – Writing Simple Programs

The chapter provides a systematic method for converting a problem statement into a Python program.

Problem – Vending Machine Change

A customer:

- selects an item;
- inserts a bill;
- receives the purchased item;
- receives change in dollar coins and quarters.

Assume item prices are multiples of 25 cents.

Step 1. Understand the Inputs and Outputs

Inputs

- bill denomination;
- item price in pennies.

Outputs

- number of dollar coins;
 - number of quarters.
-

Step 2. Work Out an Example by Hand

Suppose:

```
Bill inserted = $5.00  
Item price    = $2.25
```

Change:

```
$5.00 - $2.25 = $2.75 = 275 pennies
```

So the machine returns:

```
2 dollar coins  
3 quarters
```

Step 3. Write Pseudocode

```
change due = 100 × bill value - item price

dollar coins = change due // 100

change due = change due % 100

quarters = change due // 25
```

Step 4. Identify Variables and Constants

Variables:

```
billValue
itemPrice
changeDue
dollarCoins
quarters
```

Constants:

```
PENNIES_PER_DOLLAR = 100
PENNIES_PER_QUARTER = 25
```

All values are integers because the computation uses floor division and remainders.

Step 5. Translate the Computation into Python

```
changeDue = PENNIES_PER_DOLLAR * billValue - itemPrice

dollarCoins = changeDue // PENNIES_PER_DOLLAR
changeDue = changeDue % PENNIES_PER_DOLLAR
quarters = changeDue // PENNIES_PER_QUARTER
```

Step 6. Add Input and Output

```
billValue = int(input("Enter bill value: "))
itemPrice = int(input("Enter item price in pennies: "))

print("Dollar coins: %6d" % dollarCoins)
print("Quarters:      %6d" % quarters)
```

Step 7. Assemble the Program

```
##
# This program simulates a vending machine that gives change.
#

PENNIES_PER_DOLLAR = 100
```

```

PENNIES_PER_QUARTER = 25

billValue = int(input("Enter bill value: "))
itemPrice = int(input("Enter item price in pennies: "))

changeDue = PENNIES_PER_DOLLAR * billValue - itemPrice

dollarCoins = changeDue // PENNIES_PER_DOLLAR
changeDue = changeDue % PENNIES_PER_DOLLAR
quarters = changeDue // PENNIES_PER_QUARTER

print("Dollar coins: %6d" % dollarCoins)
print("Quarters:      %6d" % quarters)

```

The method is general:

Understand

- Calculate by hand
- Pseudocode
- Variables/constants
- Python computation
- Input/output
- Complete program

Worked Example 2.2 – Computing the Cost of Stamps

A stamp vending machine receives dollar bills and dispenses:

- first-class stamps;
- penny stamps as change.

Assume the first-class stamp price is 49 cents, as in the textbook example.

Input

- number of dollars inserted.

Outputs

- number of first-class stamps;
- number of penny stamps.

Hand Calculation for \$1

```

100 // 49 = 2 first-class stamps
100 - 2 × 49 = 2 cents change

```

General Computation

```

FIRST_CLASS_STAMP_PRICE = 49

firstClassStamps = 100 * dollars // FIRST_CLASS_STAMP_PRICE
change = 100 * dollars - firstClassStamps * FIRST_CLASS_STAMP_PRICE

```

Complete example:

```
FIRST_CLASS_STAMP_PRICE = 49

dollars = int(input("Enter number of dollars: "))

firstClassStamps = 100 * dollars // FIRST_CLASS_STAMP_PRICE
change = 100 * dollars - firstClassStamps * FIRST_CLASS_STAMP_PRICE

print("First class stamps: %6d" % firstClassStamps)
print("Penny stamps:      %6d" % change)
```

This example reinforces:

- named constants;
 - integer arithmetic;
 - floor division;
 - input conversion;
 - formatted output.
-

Computing & Society 2.2 – Bugs in Silicon

Not all incorrect numerical results are caused by application programmers.

The chapter discusses the historical **Pentium floating-point division bug**. A defect in processor hardware caused a very small set of floating-point calculations to produce incorrect results.

The broader lesson is important:

- software depends on hardware;
- hardware itself can contain defects;
- numerical computing relies on carefully designed standards and implementations;
- testing can reveal problems at multiple levels of a computing system.

The chapter also notes later processor-security issues related to speculative execution, reinforcing the idea that sophisticated optimizations can introduce unexpected risks.

Self Check 2.5

Question 1

What type does `input()` return?

Answer

A string (`str`).

Question 2

How can an integer be read directly from user input?

Answer

```
value = int(input("Enter a value: "))
```

Question 3

What does `%.2f` mean in the chapter's formatting syntax?

Answer

Display a floating-point value with two digits after the decimal point.

Question 4

Why should numerical input be converted immediately?

Answer

It avoids repeated conversions and reduces the chance of accidentally using a string in arithmetic.

Question 5

Which operators are particularly useful when computing change in whole coin units?

Answer

Floor division `//` and remainder `%`.

TOOLBOX 2.1 – Symbolic Processing with SymPy

This optional section illustrates the broader Python ecosystem.

SymPy is a package for symbolic mathematics. Instead of only computing numeric values, it can manipulate mathematical expressions symbolically.

Typical setup:

```
from sympy import *
```

Create a symbolic expression:

```
f = sympify("x ** 2 * sin(x)")
```

SymPy can perform tasks such as:

Expanding Expressions

```
expand((x - 1) * (x + 1))
```

Solving Equations

```
solve(x ** 2 + 2 * x - 8)
```

Differentiation

```
diff(f)
```

Integration

```
g = integrate(f)
```

Substitution and Numerical Evaluation

```
result = g.subs(x, 0).evalf()
```

Plotting

```
plot(x ** 2)
```

The main lesson is not to memorize all SymPy commands. It is to recognize that Python packages provide specialized expertise that can be reused inside Python programs.

Chapter 2 Summary

Variables

- A variable is a named storage location.
- Assignment stores a value in a variable.
- A variable is created when first assigned.
- Reassignment replaces the previous value.
- = is assignment, not mathematical equality.
- Important numeric types include `int` and `float`.
- Use descriptive names.
- Use uppercase names for constants according to the chapter convention.
- Use comments to explain intent.

Arithmetic

Important operators:

```
+ - * / // % **
```

Remember:

```
/ → normal division  
// → floor division  
% → remainder  
** → power
```

Useful functions:

```
abs()  
round()  
min()  
max()
```

The `math` module provides additional functions such as:

```
sqrt()  
sin()  
cos()  
log()
```

Problem Solving

Before coding:

Choose concrete values

- solve by hand
- identify the general pattern
- write the algorithm
- translate into Python

Strings

- Strings are sequences of characters.
- `len()` returns string length.
- `+` concatenates strings.
- `*` repeats a string.
- `str()`, `int()`, and `float()` perform conversions.
- String indices begin at 0.
- Methods such as `upper()`, `lower()`, and `replace()` return new strings.

Input and Output

- `input()` returns a string.
- Convert numeric input using `int()` or `float()`.
- Convert as soon as practical after reading the input.
- The chapter uses the `%` string-format operator to control output layout.

Graphics

- Create a graphics window.
 - Obtain the canvas.
 - Draw shapes and text on the canvas.
 - The origin is at the upper-left corner.
 - Color can be set by name or RGB components.
-

Summary Quiz

Question 1

What does an assignment statement do?

- A. Tests equality
- B. Stores a value in a variable
- C. Prints a value
- D. Creates a comment

Answer

B

Question 2

Which value has type `float`?

- A. 6
- B. 0
- C. 6.0
- D. -4

Answer

C

Question 3

What is the result of:

17 // 5

- A. 2
- B. 3
- C. 3.4
- D. 5

Answer

B

Question 4

What is the result of:

```
17 % 5
```

- A. 2
- B. 3
- C. 5
- D. 17

Answer

A

Question 5

What does this expression produce?

```
"Py" + "thon"
```

Answer

Python

Question 6

Given:

```
s = "Hello"
```

what is `s[1]`?

- A. "H"
- B. "e"
- C. "l"
- D. Error

Answer

B. "e"

Question 7

What type does `input()` return?

- A. Always `int`
- B. Always `float`
- C. `str`
- D. Depends on what the user types

Answer

C. `str`

Question 8

Which statement reads a floating-point value from the user?

- A. `price = input()`
- B. `price = float(input("Price: "))`
- C. `price = str(input())`
- D. `price = print(input())`

Answer

B

Question 9

Which is the best reason to use a named constant?

- A. It makes the program run faster in every case.
- B. It makes an unexplained numeric value easier to understand and change.
- C. It prevents Python from changing the value.
- D. It replaces comments entirely.

Answer

B

Question 10

What should usually happen before translating a non-trivial computation into Python?

- A. Choose random syntax.
- B. Work through at least one example by hand.
- C. Install a graphics package.
- D. Convert every value to a string.

Answer

B

Review Exercises

Exercise 1. Trace Assignments

Determine the final value of `mystery`:

```
mystery = 1
mystery = 1 - 2 * mystery
mystery = mystery + 1
```

Answer

```
Initial: mystery = 1
Second statement: mystery = 1 - 2 × 1 = -1
Third statement: mystery = -1 + 1 = 0
```

Final value: 0.

Exercise 2. Translate Mathematics into Python

Write Python expressions for:

1. The average of a and b.
2. $a^2 + b^2$.
3. $\sqrt{x^2 + y^2}$.
4. $PV(1 + r/100)^n$.

Sample Answer

```
(a + b) / 2

a ** 2 + b ** 2

sqrt(x ** 2 + y ** 2)

PV * (1 + r / 100) ** n
```

Exercise 3. Floor Division and Remainder

Given:

```
n = 1729
```

predict:

```
n % 10
n // 10
n % 100
n % 2
```

Answer

```
9
172
29
1
```

Exercise 4. String Expressions

Given:

```
s = "Hello"
t = "World"
```

find the values of:

```
len(s) + len(t)
s[1] + s[2]
s[len(s) // 2]
s + t
t + s
s * 2
```

Answer

```
10
el
l
```

```
HelloWorld  
WorldHello  
HelloHello
```

Exercise 5. Explain the Difference

Explain the differences among:

```
2  
2.0  
'2'  
"2"  
"2.0"
```

Answer

- 2 is an integer.
 - 2.0 is a floating-point number.
 - '2' and "2" are strings containing one character.
 - "2.0" is a string containing three characters.
-

Exercise 6. Roundoff Error

Consider:

```
purchase = 19.93  
payment = 20.00  
change = payment - purchase  
print(change)
```

Explain why a result such as 0.070000000000000028 may appear.

Answer

Many decimal fractions cannot be represented exactly in binary floating-point form. The internal approximations can produce a tiny numerical difference. Format or round the result when displaying money.

Practice Exercises

Exercise 1. Paper Dimensions

A letter-size sheet is 8.5 × 11 inches. There are 25.4 millimeters per inch.

Write a program that displays its dimensions in millimeters. Use named constants and comments.

Exercise 2. Powers of a Number

Read a number and display:

- its square;
- its cube;
- its fourth power.

Use ** only for the fourth power.

Exercise 3. Two Integers

Read two integers and print:

- sum;
 - difference;
 - product;
 - average;
 - absolute difference;
 - maximum;
 - minimum.
-

Exercise 4. Split a Five-Digit Integer

Read a five-digit positive integer and display its digits separately.

Example:

Input: 16384

Output: 1 6 3 8 4

Use floor division and remainder.

Exercise 5. First, Middle, and Last Character

Read a word and print:

- first character;
- last character;
- character in the middle.

For an even-length word, use the character immediately before the middle.

Exercise 6. Monogram

Read a three-part name such as:

Harold James Morgan

and display:

HJM

First write pseudocode before attempting Python.

Exercise 7. Vending Machine

Modify the vending-machine algorithm so that change is returned using:

- quarters;
- dimes;
- nickels.

Assume the item price is a multiple of 5 cents.

Exercise 8. Temperature and Dew Point

Using the formula and constants provided by your instructor or the textbook exercise, write a program that reads temperature and relative humidity and computes an approximate dew point.

Focus on:

- importing `log` from `math`;
 - translating the mathematical formula carefully;
 - using parentheses correctly.
-

Open Exercises

Exercise 1

Give three examples of **magic numbers** that might appear in a real business program. Replace each with a meaningful named constant.

Exercise 2

Explain why `=` in Python should be read as “assign” rather than “equals”. Give an example that would be impossible as an algebraic equation but valid as a Python assignment.

Exercise 3

Choose an everyday quantity that naturally requires:

- floor division;
- remainder.

Describe a program that uses both operators.

Examples include:

- converting seconds to minutes and seconds;
- converting cents to dollars and cents;
- packing items into complete boxes.

Exercise 4

Choose a short string and show every character together with its index.

Key Terms

Term	Meaning
Variable	Named storage location
Assignment	Storing a value in a variable
Assignment operator	<code>=</code>
Initialization	Giving a variable its first value
Data type	Determines representation and valid operations
<code>int</code>	Integer type
<code>float</code>	Floating-point type

Term	Meaning
Literal	Value written directly in source code
Variable name	Identifier used to refer to a variable
Constant	Value intended to remain unchanged
Magic number	Unexplained numeric literal in code
Comment	Human-readable explanation ignored by Python
Operator	Symbol that performs an operation
Expression	Combination of values, variables, operators, and calls
Precedence	Rules controlling evaluation order
Exponentiation	Power operation, **
Floor division	Division with //
Modulus / remainder	% operation
Function	Reusable operation that can receive arguments and return a value
Standard library	Built-in collection of reusable modules
Module	Group of related functions and data definitions
Roundoff error	Small numerical error from finite floating-point representation
String	Sequence of characters
String literal	String written directly in source code
Concatenation	Joining strings with +
Repetition	Repeating a string with *
Index	Position of a character in a string
Method	Operation associated with an object
Escape sequence	Backslash sequence representing a special character
Unicode	Character encoding standard for world writing systems
Prompt	Message that tells the user what input is expected
Input	Data supplied to a program
Output	Result produced by a program
Format specifier	Pattern controlling output representation
Canvas	Drawing area inside a graphics window
RGB	Red-green-blue color representation
Bounding box	Rectangle used to position an oval or other object

Study Guidance

After Chapter 2, students should be able to write small programs that manipulate numeric and text data. The most important habit is not memorizing isolated syntax, but following a disciplined process:

Understand the task

- choose meaningful variables and constants
- work through an example by hand
- write the computation clearly
- obtain input
- convert it to the correct type
- process the data
- format and display the output
- test with simple values
- inspect unexpected results
- improve the program

A useful practice cycle is:

Predict → Calculate by hand → Write → Run → Explain → Modify → Practice